Lab 18: One analyze_document tool does three jobs badly

Topic 2: Tool Design & MCP Integration · Task 2.1: Design effective tool interfaces with clear descriptions and boundaries

Runs on Simulator 6: Structured Data Extraction · Run in class

Source: SkillCertPro practice exam Part 1, Q11 (unofficial third-party question)

The question

The current implementation exposes one client tool named analyze_document with parameters for document text, requested operation, and desired format. In evaluation runs, Claude sometimes uses it to produce summaries when the downstream validator expected extracted fields, and sometimes attempts claim verification without passing the source excerpt needed for validation. What redesign best improves tool selection and downstream reliability without creating unnecessary tool sprawl?

A. Keep analyze_document and add more operation names to its schema enum, while requiring Claude to choose the exact operation before each call.

B. Replace analyze_document with a few non-overlapping tools for distinct workflows, each with clear descriptions, JSON Schema inputs, and minimal stable result fields.

C. Create a separate tool for every document field, then merge all field-level tool results into one validated downstream record.

D. Move the operation instructions into the system prompt and keep the same tool schema to reduce changes to the integration.

Simulate it live

```
# In the browser: open Simulator 6, then Lab 18
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 18

# In a terminal, from simulators/simulator-6-structured-extraction/
uv run extract.py --case UC-11
```

What you should see

The generic tool drifts into summaries. The split set calls extract_invoice with valid fields.

evidence/cases/UC-11.json shows the generic tool producing summaries or missing fields on some documents, while the split tool set calls extract_invoice with a schema-valid payload on every document.

Open `simulators/simulator-6-structured-extraction/evidence/` and find it yourself.

The answer

B. A few non-overlapping tools with clear descriptions and JSON Schema inputs.

Split an overly generic tool into a small number of purpose-specific tools that don't overlap, such as extraction, summarisation and source-backed verification. Each description should say what the tool does, when to use it and when not to, what the parameters mean, and any caveats. Descriptions are a primary driver of tool selection. The input_schema is the contract for what Claude sends, so a verification tool can require the source excerpt. strict: true can enforce schema conformance, but semantic validation is still needed downstream.

Why the others fail

A. Adding enum values keeps incompatible workflows hidden behind one catch-all tool, so the selection ambiguity remains.

C. One tool per field is tool sprawl: more tools, more selection ambiguity.

D. Leaves the interface unchanged. The schema still allows verification without an excerpt.

Check yourself

- Learner can name the three workflows that were hiding behind the operation parameter and explain why they do not overlap.
- Learner points to the description text that says when NOT to use each tool.
- Learner explains why a schema-valid extract_invoice payload still needs the semantic total check.

Now change one thing

Change DOC-04's printed total in fixtures/invoices.txt, rerun, and watch by_document_type.invoice fall while the others hold.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.